# ASIC Implementation of AES

P.V. Sriniwas Shastry, Amruta Kulkarni
Department of Electronics and Telecommunicaitons
Cummins College of Engineering for Women
Pune, INDIA

Mukul S.Sutaone
Department of Electronics and Telecommunication
College of Engineering
Pune, INDIA

*Abstract*— **This paper proposes a low power implementation of rolled architecture for AES encryption and decryption. The design employs key expansion for all the three standard key lengths of 128, 192 and 256 bits. The keys are expanded once and stored in a memory while the encryption process is carried out. We have used 180nm standard cell library to implement the design. The design was clocked at 125 MHz to obtain a throughput of 1.6Gbps for 128 bit key, 1.33Gbps for 192 bit key and 1.14Gbps for 256 bit key. In total, 58445 gates were employed to implement all key size encryption, decryption and key expansion with a very low power consumption of 22.85mW.**

**Keywords— AES, Rolled architecture, ASIC**

## I. INTRODUCTION

The rapidly growing digital applications and number of Internet users has led to an increasing demand for security measures and devices to protect the user data. Cryptography is one of the basic countermeasures against system attacks. Two types of cryptographic systems are mainly used for cryptographic purposes, the symmetric key cryptographic system and asymmetric key cryptographic system. The symmetric key cryptography, also known as private key cryptography, uses a same key for encryption and decryption. AES and DES are examples of algorithms using symmetric key cryptographic system. The asymmetric key cryptographic uses different keys for encryption and decryption. RSA and Elliptic curve cryptography are examples of algorithms using asymmetric key cryptography.

Advanced Encryption Standard (AES) [1] can be implemented in both hardware and software. Hardware implementation of AES is more reliable against the attacks than the software implementation. References [2] and [3] focus on ASIC realisation of AES by using the rolled architecture. They have calculated keys on the fly. Reference [2] achieves a throughput of 3.84Gbps whereas [3] achieves a throughput of 1.82Gbps. [4] Implements AES as a CMOS core, getting a throughput of 1Gbps at a clock frequency of 100MHz. Using a combination of T-Box and Twisted Binary Decision Diagram, [6] achieves a throughput of 10Gbps. Composite field arithmetic for Sbox is used in [7] and this achieves a gate count of 19.5K NAND gate equivalents. Reference [8] uses T-Box for combining InvSubBytes and InvMixColumns to achieve a throughput of 2.12Gbps at 166 MHz frequency.

This paper discusses the design of an encryption/decryption core for AES. The hardware resources like Sbox are shared. The core supports all three standard key lengths of AES. A novel architecture has been proposed and implemented for the Key expansion. It computes round keys of all key sizes which are stored in the memory. These keys are then read from the memory while encryption and decryption for performing Add-round-key operation. The implementation consumes low power and generates the cipher text at a high throughput. A gate count of 58445 gates is obtained for the whole implementation in our design using TSMC's 0.18µm technology. The rest of the paper is organised as follows. Section II describes the basic AES algorithm. Section III describes our implemented architecture. Section IV describes the implementation results and comparison of results.

## II. AES ALGORITHM

The AES is a 128-bit symmetric block cipher. It operates on 128-bit of a block of data. The key sizes supported by AES are 128 bits, 192 bits and 256 bits. The AES is an iterative algorithm where in the data is passed iteratively to a round function that is executed multiple times. The number of rounds (Nr) depends on the key size. Table I shows the number of rounds required for specific key sizes. Figure 1 shows the basic encryption architecture.

The 128-bit data block is divided into 16 bytes. These bytes are organised to a 4X4 array called State. The State undergoes all the operations of the AES algorithm. One round of the AES transformation consists of four different transformations — SubByte, ShiftRow, MixColumn and AddRoundKey. The four transformations are described below:

### *A. SubByte*

Subbyte is a non linear byte substitution that operates independently on each byte of the State using a substitution table (the S box). The Sbox is an invertible function that performs two transformations: A multiplicative inverse in GF ($2^8$) with polynomial $m(x) = (x^8+x^4+x^3+x+1)$ and an affine transformation (over GF (2)).

### *B. ShiftRow*

The ShiftRow transformation consists of a cyclic shift, with different offsets on the rows of the state. While the first row of the state is not shifted, the second row of the state is shifted left cyclic by one byte, third row shifted left cyclic by two bytes and similarly the last row by three bytes.

### C. *MixColumn*

The MixColumn transformation operates on the State column-by-column. The columns are considered as polynomials over GF ($2^8$) and multiplied with a fixed polynomial modulo $x^4+1$.

### D. *AddRoundKey*

AddRoundKey is an XOR transformation which adds round key to the State in each round. The keys are expanded during the key expansion phase and kept ready for this operation.

The encryption begins with XOR of plain text and input key as initial round. After this initial round, the algorithm proceeds by applying the round function consisting of the four different transformations SubByte, ShiftRow, MixColumn and AddRoundKey. In the last round, the MixColumn transformation is skipped. The Decryption procedure of the AES is basically the inverse of each transformation (InvSubByte, InvShiftRow, InvMixColumn and AddRoundKey) in the reverse order. Similar to encryption, InvMixColumn is skipped in the last round, in decryption.

TABLE I. BLOCK LENGTH, KEY LENGTH AND NUMBER OF ROUNDS

| | **Block length (Nb words)** | **Key Length (Nk words)** | **No. Of Rounds (Nr)** |
|---|---|---|---|
| AES-128 | 4 | 4 | 10 |
| AES-192 | 4 | 6 | 12 |
| AES-256 | 4 | 8 | 14 |

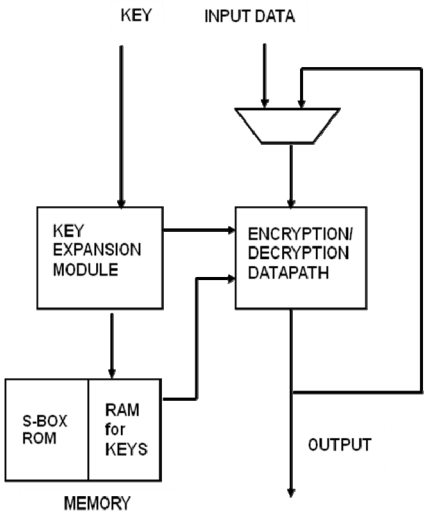


Figure 1. Basic Architecture

## III. IMPLEMENTED ARCHITECTURE

In this paper, encryption and decryption of the data is done using rolled architecture. Refer Figure 1. Once the key and data are available at the input, the key expansion module expands this key to generate the keys required for encryption/decryption, and these are then stored in the memory and are retrieved during encryption or decryption phase. To perform an encryption/decryption, data from input port and keys from memory are made available at the encryption/decryption datapath. The encryption/ decryption procedure is executed whenever a 128-bit data block is available at the input port and strobe is applied to indicate the start of encryption. The AES round function is applied for 10, 12 or 14 times depending on the key size. Finally the processed data is retrieved through the external data bus. Figure2 shows the Key Expansion module and Figure3 shows the encryption and decryption datapath.

### A. *Encryption/Decryption Datapath:*

The Encryption/Decryption Datapath consists of the individual modules, which make up the AES architecture. The operations performed during decryption are the inverse of the operations performed during encryption. The four transformations in AES encryption/decryption are as follows

*SubByte/InvSubByte*
*ShiftRow/InvShiftRow*
*MixColumn/InvMixColumn*
*AddRoundKey*

The SubByte and ShiftRow transformations are combined into one operation [9]. As mentioned earlier, SubByte transformation is performed by substituting the input byte by a value from a table known as the Sbox. The output of this transformation is then placed at the predefined positions as per the ShiftRow transformation. Thus we are performing SubByte and ShiftRow as a single stage operation. The Sbox is stored in ROM and is duplicated 16 times to perform simultaneous operation. Similar operation is performed for decryption. Like Sbox, inverse Sbox is stored in ROM and is duplicated 16 times, and is used for substitution and shifting is done in the opposite direction. In the MixColumn transformation, the State array is multiplied by a standard matrix which consists of elements 1, 2, and 3. The InvMixColumn transformation involves multiplication of the state array by a standard matrix consisting of the elements 0E, 0B, 0D and 09.

The AddRoundKey transformation consists of XOR-ing of the output of the MixColumn with the corresponding round key obtained from the key expansion routine. For each round of the main loop, a round key is derived from the original key through a process called the Key Expansion.

### B. *Key Expansion Module*

The design of the Key Expansion Unit is as shown in the Figure 2. The Key Expansion Module generates Nb (Nr + 1) words from the input cipher key, where Nb is the block length and Nr is the number of rounds. Based on the key size (128,192 or 256 bits), the number of rounds can be 10, 12 or14 and the size of the round keys can be 1408, 1664 or 1920 bits. The key schedule consists of a linear array of 4-byte words, denoted by w[i], where i is in the range of 0 to Nb (Nr + 1) .

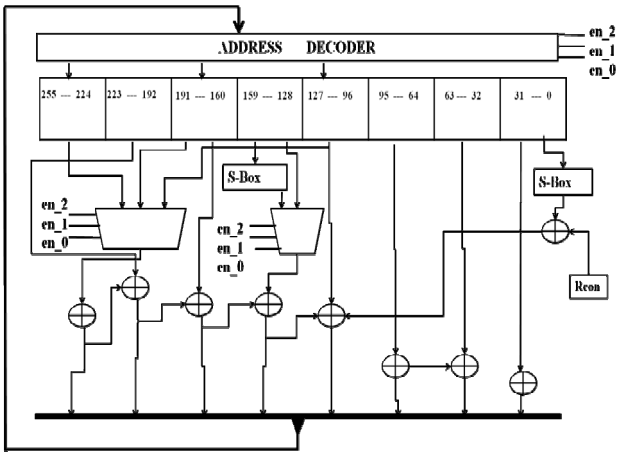


Figure 2. Key Expansion Module

These keys are calculated by the Key Expansion Module, and are simultaneously stored in a memory, for further use.

The architecture of the Key Expansion Module is as shown in Figure 2. The Key Expansion Module uses 4 or 8 S-Boxes simultaneously depending upon the key size and one ROM unit for providing the round constant (Rcon) required in the key expansion process. The signals en_2, en_1 and en_0 act as select lines to the address decoder, to write the data into the register, and to the multiplexer, for reading out the data from the register. Depending on the key size, outputs of the corresponding S-Boxes are selected. The configuration of key size is done based on the values of en_0, en_1 and en_2 signals. The truth table for key size determination based on these enable signals are given in the Table II. Whenever a key is changed, it is stored in a 256 bit register. For 128 bit or 192 bit key, upper 128 or 64 bits respectively, are set to zero. Then the Key Expansion Module operates on this key. One key is expanded per clock cycle. Hence, 10 clock cycles are required for 128 bit key, 12 clock cycles for 192 bit key and 14 clock cycles for 256 bit key.

TABLE II. KEY SIZE SELECTION TRUTH TABLE

| en_2 | en_1 | en_0 | Key Size |
|---|---|---|---|
| 0 | 0 | 1 | 128 |
| 0 | 1 | 1 | 192 |
| 1 | 0 | 0 | 256 |

The key expansion unit works in two configurations: namely as *concurrent encryption and roundkey computation (CERC)* and *pre-encryption roundkey computation (PERC)*. In the PERC configuration the key expansion is performed first and all the round keys are computed and stored in the memory. Once the key expansion is over encryption rounds are started in the next clock cycle. The S-Boxes required in this configuration are less compared to CERC as S-Boxes required for encryption and key expansion could be common. This makes the merit of this configuration but increases the latency. The implementation has achieved latencies of 21, 21 and 23 for 128, 192 and 256 bit key encryption respectively

In CERC configuration the key expansion is concurrently done while encryption round is performed. This configuration shares the same clock time for computation of encryption round. Unlike on the fly key expansion, this method is performed only once when a new key is presented at the input port. The expanded round keys are stored in the memory and for the next block of plain text; key expansion need not be performed. Since the encryption is performed concurrently with the key expansion, the latency is of 10 clock cycles for 128 bit key, 12 clock cycles for 192 bit key and 14 clock cycles for 256 bit key. The S-Boxes required for the Key Expansion Module are the same used in the encryption process. This increases the sharing of hardware resources. The keys are calculated and encryption is carried out simultaneously for the first set of data and the keys are also simultaneously stored in a memory. For the successive set of data, the keys are retrieved from the memory. For decryption, if the key is same, the expanded keys are retrieved from the

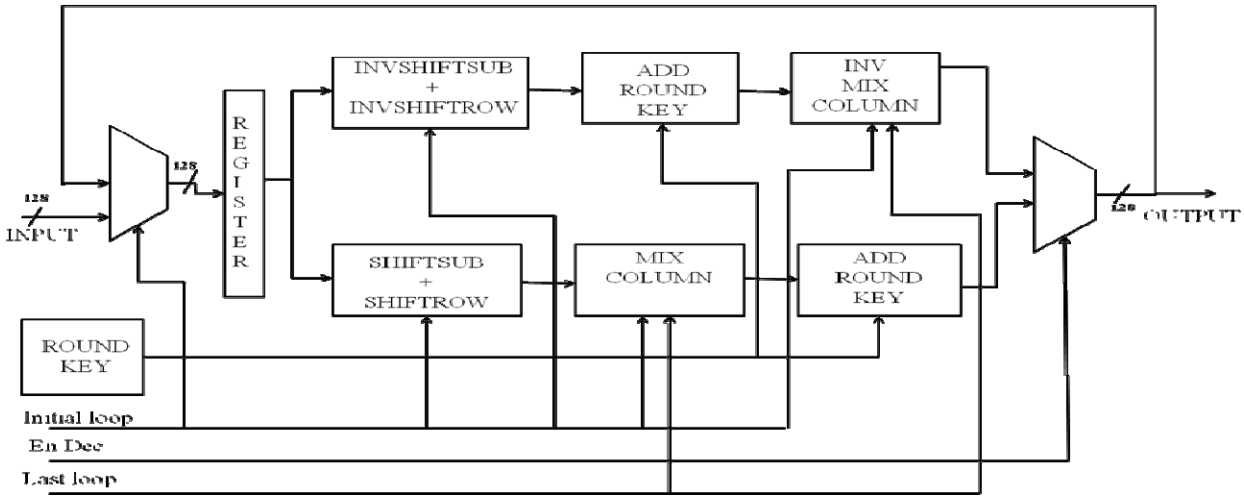


Figure 3. Encryption/Decryption Datapath

memory. If the key has changed at the input, the keys are expanded first and then the decryption process is carried out.

## IV. IMPLEMENTATION RESULT

The design is described using VHDL language. The simulation is done with Cadence NCSim and compiled using RTL compiler v 10.1.100. A throughput of 1.6Gbps for 128 bit key size, 1.33Gbps for 192 bit key size and 1.14Gbps for 256 bit key size is obtained at the clock frequency of 125MHz. The maximum clock frequency is obtained by the following formula mentioned in [9]:

$$t_{clk} = t_{\delta} + \max [(t_{subshift} + t_{mixcol} + t_{addroundkey}), t_{keyexpansion}]$$

Key expansion unit requires a time of 3.84 ns for expanding the keys, while the encryption-decryption unit requires 6.1 ns for encryption. The throughput is calculated using the formula given below:

Throughput = (Block Size * Clock Frequency) / (number of clock cycles needed to process one block of data)

The implementation used a total of 58445 gates. The breakdown of the gates is given in the Table III. The total power obtained was 22.85mW, out of which dynamic power is 21.79mW and the static power is 1.057mW. The breakdown of power is given in Table IV.

The comparison of the results is mentioned in Table V. There are other rolled architecture implementations and their results published in [2], [3], [4], [5], [6], [7] and [8]. Reference [2] achieves a better throughput than our design but it supports only encryption. Also the round keys are calculated on-the-fly. So the key scheduling unit is active all the time. This increases the power consumption. The power consumed is 54mW at 50MHz. The rolled architecture implementation of [3] achieves a throughput of 1.82Gbps at a maximum frequency of 100MHz. It required 173K gates to implement for only encryption. Iterative architecture of [4] achieves a throughput of 1Gbps at 100MHz with power consumption more than our implementation. Using a combination of T-Box and Twisted Binary Decision Diagram, [6] achieves a throughput of 10Gbps. Composite field arithmetic for Sbox is used in [7] and this achieves a gate count of 19.5K NAND gate equivalents. Reference [8] uses T-Box for combining InvSubByte and InvMixColumn to achieve a throughput of 2.12Gbps at 166MHz frequency requiring a of total 119K gates. Our design requires a total of 58.445K to implement encryption, decryption and key expansion all inclusive. This gate count is also inclusive of the ROM required for the S-Boxes and RAM required for storing the expanded keys. Thus, in comparison to other papers referred, our design requires less area, in terms of number of gates, while power consumption is the least out of all ROM based implementation of the S-Box.

The InvMixColumn has been implemented by multiplying with the matrix consisting of byte values 09, 0B, 0D and 0E [1]. A different approach to the implementation of InvMixColumn would be to factorize the terms 09, 0B, 0D and 0E in terms of 04, 08 and matrix used for MixColumn Operation. By doing so, the hardware required for MixColumn in encryption can be used to implement InvMixColumn [9].

For the CERC and PERC configuration of key expansion the gate count is almost same but there is difference in power consumption. The power consumption increases whenever new key is available at input port, because the key expansion has to compute new round keys. The key expansion is not performed for every new plain text block unless and until new key is provided and stored round keys are used.

## V. CONCLUSION

This paper presented a rolled architecture implementation of AES using 180nm standard cell library, supporting all key sizes. The design could work at 125MHz and a throughput of 1.6Gbps, 1.33Gbps and 1.14Gbps was obtained for a key size of 128,192 and 256 bits respectively, for 128 bits block size. A total of 58445 gates were employed for the implementation of the entire design. Our design has the highest Throughput/gate ratio as compared with the designs using ROM based S-Box and implementation with both encryption and decryption. A low power consumption of 22.58mW is achieved for the entire implementation of encryption, decryption, and PREC type key expansion unit with latencies of 21, 21 and 23 for 128, 192 and 256 bit key sizes respectively. The power consumption reduces to 20.58mW for rest of the encryption when no new key is being processed.

TABLE III. TOTAL GATES REQUIRED FOR INDIVIDUAL UNITS

| | |
|---|---|
| Total | 58445 gates |
| Key Expansion Unit | 8753 gates |
| Encryption +Decryption Unit | 19796 gates |
| SubByte + ShiftRow/ InvSubByte + InvShiftRow | 15288 gates |
| MixCol / InvMixCol | 1662 gates |
| AddRoundKey | 2050 gates |
| Memory(RAM) | 28635 |

TABLE IV. TOTAL POWER AND POWER DISTRIBUTION FOR INDIVIDUAL UNITS

| Total | 22.85mW |
|---|---|
| Key Expansion Unit | 2.322mW |
| Encryption + Decryption Unit | 6.355mW |
| SubByte + ShiftRow/ InvSubByte + InvShiftRow | 2.753mW |
| MixCol + InvMixCol | 0.8335mW |
| AddRoundKey | 1.734mW |
| Memory | 10.56mW |

TABLE V. COMPARISON OF RESULTS

| | Our | [2] | [3] | [4] | [6] | [7] | [8] |
|---|---|---|---|---|---|---|---|
| Technology | 0.18µm | 0.18µm | 0.18µm | 0.18µm | 0.18µm | 0.18µm | 0.25µm |
| Architecture | Rolled | Rolled | Rolled | Rolled | Rolled | Rolled | Rolled |
| Gate count | 58.445K | -- | 173K | -- | 173K | 19.5K | 119K |
| Clock Rate | 125MHz | 330MHz | 100MHz | 100MHz | 125MHz | 100MHz | 166MHz |
| Power | 22.85mW | 54mW@50MHz | -- | 110.037mW | 56mW | -- | 600mW |
| Encryption/Decryption | Both | Only Enc | Only Enc | Both | Only Enc | Both | Both |
| Key Sizes | All | Only 128 bit | All | Only 128 bit | All | All | All |
| Key Expander/Scheduler | Yes | Yes | Yes | Yes | No | On-the-fly key expansion | Two on-the fly key generators |
| Throughput(Gbps) | 1.6 | 3.84 | 1.82 | 1.0 | 10 | 1.16 | 2.12 |
| Throughput/gate count(Kb/s gates) | 27.48196 | -- | 10.52023 | -- | 57.803 | 59.487 | 17.81512 |

## REFERENCES


[1] Specification for the Advanced Encryption Standard (AES) Federal Information Processing Standards (FIPS) Publication 197 http://csrc.nist.gov/encryption/aes/frn-fips197.pdf (Nov -2001)

[2] Alireza Hodjat, David D. Hwang, Bocheng Lai, Kris Tiri, Ingrid Verbauwhede, "A 3.84Gbits/s AES Crypto Coprocessor with Modes of Operation in a 0.18µm CMOS Technology", GLSVLSI'05, April 17-19, 2005.

[3] Henry Kuo, Ingrid Verbauwhede, "Architectural Optimization for a 1.82Gbits/sec VLSI Implementation of the AES Rijndael Algorithm", LNCS, 2001, Volume 2162,Cryptographic Hardware and Embedded Systems – CHES 2001, pp. 51-64.

[4] Lan Liu and David Luke, "Implementation of AES as a CMOS core", IEEE Canadian Conference on Electrical and Computer Enigneering, 2003.

[5] Ingrid Verbauwhede, Patrick Schaumont and Henry Kuo, "Design and Performance Testing of a 2.29-GB/s Rijndael Processor", IEEE Journal of Solid-State Circuits, Vol.38, NO.3, March 2003, pp. 569-572.

[6] Sumio Morioka and Akashi Satoh, "A 10-Gbps Full-AES Crypto Design With a Twisted BDD S-Box Architecture", IEEE Transactions On Very Large Scale Integration (VLSI) Systems, Vol. 12, No. 7, July 2004, pp. 686-691.

[7] Qingfu Cao, Shuguo Li, "A High-Throughput Cost-Effective ASIC Implementation of the AES Algorithm", IEEE 8th International Conference on ASIC, 2009, pp.805-808.

[8] F. K. Gürkaynak, A. Burg, N. Felber, W. Fichtner D. Gasser, F. Hug, H. Kaeslin, "2 Gb/s Balanced AES Crypto-Chip Implementation", GLSVLSI'04, April 26–28, 2004, Boston, Massachusetts, USA.

[9] P.V.Sriniwas Shastry, Priya Deshpande, M.S.Sutaone, "A Sub-Pipelined Implementation of AES for all key sizes", International Journal of Advances in Computer Networks and its Security, Volume2, issue 2 ,page 37-41.